

Programmierung

–

Level

1.4

1.	Vorwort zu den Programmierübungen	3
2.	Ziel der Übung.....	3
3.	Was ist auf der ATC-Platine bereits vorbereitet?.....	4
4.	Neue Befehle/Konzepte in dieser Übung	4
5.	Erklärung der verwendeten Befehle	5
6.	Schritt-für-Schritt: Schreibe den Code selbst	6
6.1.	Schritt 1 – Pins festlegen	6
6.2.	Schritt 2 – setup(): Ein- und Ausgänge konfigurieren	6
6.3.	Schritt 3 – Aufgabe A: Taster steuert LED direkt	7
6.4.	Schritt 4 – Aufgabe B: LED Zustand im Seriellen Monitor anzeigen	7
6.5.	Schritt 5 – Aufgabe C: Helligkeit mit dem Potentiometer steuern	8
7.	Challenge	9
8.	Typische Fehler & schnelle Diagnose	11
9.	Referenzlösung Aufgabe A	12
10.	Referenzlösung Aufgabe B	14
11.	Referenzlösung Aufgabe C.....	16
12.	Referenzlösung Challenge	19
13.	Abschluss	22

1. Vorwort zu den Programmierübungen

Die folgenden Programmierübungen sind so aufgebaut, dass sie Schritt für Schritt in die Arbeit mit Mikrocontrollern einführen.

Alle Übungen können mit dem Hardware-Einsteigerpaket Level 1 umgesetzt werden und bauen inhaltlich aufeinander auf.

Zu Beginn werden grundlegende Konzepte vermittelt und einfache Aufgaben umgesetzt. Mit jeder weiteren Übung werden neue Funktionen ergänzt und bestehende Inhalte erweitert.

Ziel ist es, ein solides Verständnis für den Aufbau von Programmen, den Umgang mit der Arduino IDE und das Zusammenspiel von Hard- und Software zu entwickeln.

Die Übungen sind bewusst übersichtlich gehalten und sollen dazu ermutigen, eigene Anpassungen vorzunehmen und mit dem Code zu experimentieren.

Es wird empfohlen, die Programmierübungen in der vorgesehenen Reihenfolge durchzuführen, da spätere Aufgaben auf den vorherigen aufbauen.

So entsteht nach und nach ein nachvollziehbarer Lernpfad, der als Grundlage für weiterführende Projekte dient.

2. Ziel der Übung

In dieser Programmierübung lernst du die Nutzung des seriellen Monitors kennen.

Du erfährst, wie der Mikrocontroller Informationen an den Computer senden kann und wie diese im Seriellen Monitor sichtbar dargestellt werden.

Zusätzlich lernst du erstmals einen analogen Eingang kennen und arbeitest nicht mehr nur mit festen Zuständen wie EIN oder AUS, sondern auch mit kontinuierlichen Messwerten von 0 bis 1023.

Diese Übung bildet die Grundlage für die spätere Arbeit mit Sensoren in Level 2.

Am Ende kannst du:

- wie der serielle Monitor gestartet wird
- wie Text und Werte ausgegeben werden
- wie Taster- und LED-Zustände angezeigt werden
- wie ein Analogwert ausgelesen wird
- wie ein Messwert eine Programmentscheidung beeinflusst

3. Was ist auf der ATC-Platine bereits vorbereitet?

Für diese Übung werden folgende Pins verwendet

- **LED 1** ist an **Pin D5**
- **LED 2** ist an **Pin D6**
- **Taster 1** ist an **Pin D7**
- **Taster 2** ist an **Pin D8**
- **Potentiometer** ist an **A0**

Du musst nichts verdrahten.

Hinweis:

Nicht alle Pins des Arduino Nano unterstützen PWM.

Für spätere Aufgaben werden deshalb bewusst D5 und D6 verwendet. Auf der ATC-Level-1-Platine sind folgende Bauteile angeschlossen:

4. Neue Befehle/Konzepte in dieser Übung

In dieser Übung verwenden wir erstmals einen Analogen Eingang und den seriellen Monitor.

Neue Befehle sind:

- `Serial.begin()`
- `Serial.print()`
- `Serial.println()`
- `analogRead()`
- Wertebereiche 0-1023
- Schwellwert vergleich (`<=`; `>=`; `==`)
- `map()`

5. Erklärung der verwendeten Befehle

pinMode(pin, INPUT_PULLUP)

Legt einen Pin als Eingang fest und aktiviert einen internen Widerstand.
Dadurch wird kein externer Widerstand benötigt.

- nicht gedrückt → HIGH
- gedrückt → LOW

Serial.begin(9600)

Startet die serielle Kommunikation zwischen Arduino und Computer.
Die Zahl 9600 ist die Übertragungsgeschwindigkeit (Baudrate).

Wichtig:

Die Baudrate im Seriellen Monitor muss identisch eingestellt sein.

Serial.print()

Gibt Text oder Zahlen aus.

Serial.println()

Wie Serial.print(),
springt danach jedoch in eine neue Zeile.

analogRead(pin)

Liest einen analogen Eingang aus.

Der Rückgabewert liegt zwischen:

- 0 (0 Volt)
- 1023 (5 Volt)

Es werden keine echten Spannungen angezeigt,
sondern digitale Messwerte.

Vergleich mit einem Schwellwert

Ein Messwert kann mit einer Zahl verglichen werden:

Beispiel:

if (wert > 500)

map()

Mit map() kann ein Zahlenwert von einem Wertebereich in einen anderen Wertebereich umgerechnet werden.

Beispiel:

int Variable_B= map(Variable_A, 0, 1023, 0, 255);

Variable_A ist der eingangswert

Variable_B die Ziel Variable in der der neu Skalierte wert abgelegt wird

6. Schritt-für-Schritt: Schreibe den Code selbst

6.1. Schritt 1 – Pins festlegen

Erstelle einen neuen Sketch und lege folgende Pins als Konstanten oder Variablen fest:

- LED 1
- LED 2
- Taster 1
- Taster 2
- Potentiometer

Aufgabe:

Nutze die Pin-Nummern 5, 6, 7, 8 und A0.

Selbstcheck:

Du hast jetzt fünf klar benannte Pins im Code.

6.2. Schritt 2 – setup(): Ein- und Ausgänge konfigurieren

In setup() musst du:

- die beiden LED-Pins als OUTPUT konfigurieren.
- die beiden Taster-Pins als INPUT_PULLUP konfigurieren.

und die Verbindung für den Seriellen Monitor initialisieren.

Selbstcheck:

Im setup() stehen jetzt vier pinMode()-Aufrufe und der Aufruf Serial.begin(9600).

6.3. Schritt 3 – Aufgabe A: Taster steuert LED direkt

Jetzt liest du die Zustände der beiden Taster ein und speicherst den aktuellen Zustand der LEDs jeweils in einer eigenen Variable.

Aufgabe:

- Beim Programmstart → beide LED-Zustände = AUS
- Taster 1 gedrückt → LED 1 Zustand wechseln
- Taster 1 erneut gedrückt → LED 1 Zustand erneut wechseln
- Taster 2 gedrückt → LED 2 Zustand wechseln
- Taster 2 erneut gedrückt → LED 2 Zustand erneut wechseln

Die gespeicherten Zustände bestimmen anschließend, ob die LEDs EIN oder AUS geschaltet werden.

Hinweis:

Denke daran, dass ein gedrückter Taster bei INPUT_PULLUP den Wert LOW liefert.

Selbstcheck:

- Jeder Tastendruck ändert den Zustand genau einmal
- Die LEDs behalten ihren Zustand nach dem Loslassen der Taster
- Die LEDs entsprechen immer ihren gespeicherten Zuständen

6.4. Schritt 4 – Aufgabe B: LED Zustand im Seriellen Monitor anzeigen

Jetzt werden die aktuellen Zustände der LEDs im Seriellen Monitor sichtbar gemacht.

Aufgabe:

Gib die Werte der LEDs im Seriellen Monitor aus.

Hinweis:

Der Serielle Monitor eignet sich ideal zum Debuggen und Kontrollieren von Programmen.

Selbstcheck:

Beim Drücken der Taster ändern sich die angezeigten Werte der LEDs sichtbar zwischen:

- 1 → AN
- 0 → AUS

6.5. Schritt 5 – Aufgabe C: Helligkeit mit dem Potentiometer steuern

Die LEDs sollen unterschiedlich hell leuchten können.
Hierzu wird der Analogwert des Potentiometers verwendet.

Aufgabe:

- Lies den Wert des Potentiometers an A0 aus
- Verwende den Potentiometerwert für die Helligkeit der LEDs
- Taster 1 schaltet die LED1 ein oder aus
- Taster 2 schaltet die LED2 ein oder aus

Hinweis:

Das Potentiometer liefert Werte von 0 bis 1023
Für die LED-Helligkeit mit PWM wird aber ein Wert von 0 bis 255 benötigt.
Der Wert muss deshalb umgerechnet werden.

Wichtig:

Die LEDs liegen auf D5 und D6.
Diese Pins unterstützen PWM und können deshalb mit `analogWrite()` für Helligkeitswerte verwendet werden.

Selbstcheck:

- Taster 1 schaltet die LED1 ein oder aus.
- Taster 2 schaltet die LED2 ein oder aus.
- Wenn die LEDs eingeschaltet sind, verändert das Potentiometer die Helligkeit
- Sind die LEDs ausgeschaltet, bleiben sie aus, auch wenn das Potentiometer gedreht wird

7. Challenge

Diese Aufgaben bauen auf der Bonusaufgabe auf und erfordern saubere Logik. Sie sind bewusst anspruchsvoll.

Aufgabe:

- Gib einen Zahlenwert im Seriellen Monitor ein
- Der Wert soll zwischen 0 und 255 liegen
- Dieser Wert bestimmt die Helligkeit der LEDs

Beispiel:

- 0 → LEDs AUS
- 50 → LEDs leuchten dunkel
- 150 → LEDs leuchten mittelhell
- 255 → LEDs leuchten maximal hell

Anforderung:

- Der eingegebene Wert wird im Programm gespeichert
- Die LEDs behalten diese Helligkeit, bis ein neuer Wert eingegeben wird
- Werte außerhalb von 0 bis 255 sollen nicht verwendet werden
- Die Taster schalten die LEDs weiterhin ein oder aus

Hilfestellung – Werte aus dem Seriellen Monitor einlesen

Der Arduino kann nicht nur Daten senden, sondern auch Daten vom Computer empfangen.

Dafür wird geprüft:

- ob neue Zeichen empfangen wurden
- und welcher Wert übertragen wurde

Wichtige Befehle:

Serial.available()

Prüft, ob Daten im Seriellen Monitor empfangen wurden.

Beispiel:

```
if (Serial.available() > 0)
```

Das bedeutet:

Es wurden Zeichen empfangen und können gelesen werden.

Serial.parseInt()

Liest eine Zahl aus dem Seriellen Monitor.

Beispiel:

```
int wert = Serial.parseInt();
```

Der eingegebene Zahlenwert wird dabei automatisch als Integer gespeichert.

Beispiel:

```
if (Serial.available() > 0) {  
  
    int helligkeit = Serial.parseInt();  
  
}
```

8. Typische Fehler & schnelle Diagnose

- **Im Seriellen Monitor passiert nichts**
Prüfe, ob `Serial.begin(9600);` in `setup()` vorhanden ist.
- **Nur Zeichenmüll sichtbar**
Prüfe, ob die Baudrate im Seriellen Monitor auf 9600 eingestellt ist.
- **Die LEDs reagieren nicht auf eingegebene Werte**
Prüfe, ob die Werte mit `Serial.parseInt()` eingelesen werden.
- **Die Helligkeit verändert sich nicht**
Prüfe, ob `analogWrite()` verwendet wird und die LEDs an D5 oder D6 angeschlossen sind.
- **Die LEDs sind immer voll hell oder komplett aus**
Prüfe, ob der Wertebereich zwischen 0 und 255 liegt.
- **Nach der Eingabe passiert erst verzögert etwas**
Prüfe, ob der Wert im Seriellen Monitor wirklich abgeschickt wurde.
- **Die LEDs reagieren trotz Eingabe nicht**
Prüfe, ob `Serial.available()` verwendet wird, bevor Daten gelesen werden.

9. Referenzlösung Aufgabe A

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.4 Aufgabe A: Taster schalten LED-Zustände um

```
// Pinbelegung der ATC-Platine
const int LED1_PIN = 5; // LED 1
const int LED2_PIN = 6; // LED 2
const int BTN1_PIN = 7; // Taster 1
const int BTN2_PIN = 8; // Taster 2
const int POT_PIN = A0; // Potentiometer

// Speichert den aktuellen Zustand der LEDs
// false -> LED aus
// true  -> LED an
bool led1State = false;
bool led2State = false;

// Sperrvariablen für die Taster
// Dadurch wird pro Tastendruck nur einmal geschaltet
bool btn1Gesperrt = false;
bool btn2Gesperrt = false;

void setup() {

    // LEDs als Ausgänge konfigurieren
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);

    // Taster als Eingänge mit interner Pull-Up-Schaltung
    // nicht gedrückt -> HIGH
    // gedrückt      -> LOW
    pinMode(BTN1_PIN, INPUT_PULLUP);
    pinMode(BTN2_PIN, INPUT_PULLUP);

    // Serielle Kommunikation starten
    Serial.begin(9600);

    // LEDs beim Start ausschalten
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, LOW);
}
```

```
void loop() {  
  
    // Tasterzustände einlesen  
    // digitalRead() liefert HIGH oder LOW  
    bool btn1Pressed = digitalRead(BTN1_PIN);  
    bool btn2Pressed = digitalRead(BTN2_PIN);  
  
    // Taster 1 wurde gedrückt  
    // und war vorher noch nicht gesperrt  
    if (btn1Pressed == LOW && btn1Gesperrt == false) {  
  
        // LED 1 Zustand umschalten  
        // aus -> an  
        // an -> aus  
        led1State = !led1State;  
  
        // Taster sperren  
        btn1Gesperrt = true;  
    }  
  
    // Sobald Taster 1 losgelassen wird,  
    // wird die Sperre wieder aufgehoben  
    if (btn1Pressed == HIGH) {  
        btn1Gesperrt = false;  
    }  
  
    // Taster 2 wurde gedrückt  
    // und war vorher noch nicht gesperrt  
    if (btn2Pressed == LOW && btn2Gesperrt == false) {  
  
        // LED 2 Zustand umschalten  
        led2State = !led2State;  
  
        // Taster sperren  
        btn2Gesperrt = true;  
    }  
  
    // Sobald Taster 2 losgelassen wird,  
    // wird die Sperre wieder aufgehoben  
    if (btn2Pressed == HIGH) {  
        btn2Gesperrt = false;  
    }  
  
    // Gespeicherten Zustand auf die LEDs ausgeben  
    digitalWrite(LED1_PIN, led1State);  
    digitalWrite(LED2_PIN, led2State);  
}
```

10. Referenzlösung Aufgabe B

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.4 Aufgabe B: LED-Zustände im Seriellen Monitor anzeigen

```
// Pinbelegung der ATC-Platine
const int LED1_PIN = 5; // LED 1
const int LED2_PIN = 6; // LED 2
const int BTN1_PIN = 7; // Taster 1
const int BTN2_PIN = 8; // Taster 2
const int POT_PIN = A0; // Potentiometer

// Speichert den aktuellen Zustand der LEDs
// false -> LED aus
// true -> LED an
bool led1State = false;
bool led2State = false;

// Sperrvariablen für die Taster
// Dadurch wird pro Tastendruck nur einmal geschaltet
bool btn1Gesperrt = false;
bool btn2Gesperrt = false;

void setup() {

    // LEDs als Ausgänge konfigurieren
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);

    // Taster als Eingänge mit interner Pull-Up-Schaltung
    // nicht gedrückt -> HIGH
    // gedrückt -> LOW
    pinMode(BTN1_PIN, INPUT_PULLUP);
    pinMode(BTN2_PIN, INPUT_PULLUP);

    // Serielle Kommunikation starten
    Serial.begin(9600);

    // LEDs beim Start ausschalten
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, LOW);
}
```

```
void loop() {  
  
    // Tasterzustände einlesen  
    bool btn1Pressed = digitalRead(BTN1_PIN);  
    bool btn2Pressed = digitalRead(BTN2_PIN);  
  
    // Taster 1 schaltet LED 1 um  
    if (btn1Pressed == LOW && btn1Gesperrt == false) {  
  
        led1State = !led1State;  
        btn1Gesperrt = true;  
    }  
  
    // Sperre wieder freigeben,  
    // sobald Taster 1 losgelassen wurde  
    if (btn1Pressed == HIGH) {  
        btn1Gesperrt = false;  
    }  
  
    // Taster 2 schaltet LED 2 um  
    if (btn2Pressed == LOW && btn2Gesperrt == false) {  
  
        led2State = !led2State;  
        btn2Gesperrt = true;  
    }  
  
    // Sperre wieder freigeben,  
    // sobald Taster 2 losgelassen wurde  
    if (btn2Pressed == HIGH) {  
        btn2Gesperrt = false;  
    }  
  
    // Gespeicherten Zustand auf die LEDs ausgeben  
    digitalWrite(LED1_PIN, led1State);  
    digitalWrite(LED2_PIN, led2State);  
  
    // LED-Zustände im Seriellen Monitor anzeigen  
    Serial.print("LED1: ");  
    Serial.print(led1State);  
  
    Serial.print(" | LED2: ");  
    Serial.println(led2State);  
  
    // Kleine Pause für bessere Lesbarkeit im Seriellen Monitor  
    delay(200);  
}
```

11. Referenzlösung Aufgabe C

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.4 Aufgabe C: Helligkeit mit Potentiometer steuern

```
// Pinbelegung der ATC-Platine
const int LED1_PIN = 5; // LED 1, PWM-fähig
const int LED2_PIN = 6; // LED 2, PWM-fähig
const int BTN1_PIN = 7; // Taster 1
const int BTN2_PIN = 8; // Taster 2
const int POT_PIN = A0; // Potentiometer

// Speichert den aktuellen Zustand der LEDs
// false -> LED aus
// true -> LED an
bool led1State = false;
bool led2State = false;

// Sperrvariablen für die Taster
// Dadurch wird pro Tastendruck nur einmal geschaltet
bool btn1Gesperrt = false;
bool btn2Gesperrt = false;

void setup() {

    // LEDs als Ausgänge konfigurieren
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);

    // Taster als Eingänge mit interner Pull-Up-Schaltung
    // nicht gedrückt -> HIGH
    // gedrückt -> LOW
    pinMode(BTN1_PIN, INPUT_PULLUP);
    pinMode(BTN2_PIN, INPUT_PULLUP);

    // Serielle Kommunikation starten
    Serial.begin(9600);

    // LEDs beim Start ausschalten
    analogWrite(LED1_PIN, 0);
    analogWrite(LED2_PIN, 0);
}
```



```
void loop() {  
  
    // Tasterzustände einlesen  
    bool btn1Pressed = digitalRead(BTN1_PIN);  
    bool btn2Pressed = digitalRead(BTN2_PIN);  
  
    // Analogwert des Potentiometers lesen  
    // Der Wert liegt zwischen 0 und 1023  
    int potValue = analogRead(POT_PIN);  
  
    // Analogwert auf PWM-Wert umrechnen  
    // 0-1023 wird zu 0-255  
    int pwmValue = map(potValue, 0, 1023, 0, 255);  
  
    // Taster 1 schaltet LED 1 um  
    if (btn1Pressed == LOW && btn1Gesperrt == false) {  
  
        led1State = !led1State;  
        btn1Gesperrt = true;  
    }  
  
    // Sperre wieder freigeben,  
    // sobald Taster 1 losgelassen wurde  
    if (btn1Pressed == HIGH) {  
        btn1Gesperrt = false;  
    }  
  
    // Taster 2 schaltet LED 2 um  
    if (btn2Pressed == LOW && btn2Gesperrt == false) {  
  
        led2State = !led2State;  
        btn2Gesperrt = true;  
    }  
  
    // Sperre wieder freigeben,  
    // sobald Taster 2 losgelassen wurde  
    if (btn2Pressed == HIGH) {  
        btn2Gesperrt = false;  
    }  
  
    // Wenn LED 1 eingeschaltet ist,  
    // wird die Helligkeit über PWM gesetzt  
    if (led1State == true) {  
  
        analogWrite(LED1_PIN, pwmValue);  
    }  
}
```

```
} else {  
  
    // Wenn LED 1 ausgeschaltet ist,  
    // bleibt sie aus  
    analogWrite(LED1_PIN, 0);  
}  
  
// Wenn LED 2 eingeschaltet ist,  
// wird die Helligkeit über PWM gesetzt  
if (led2State == true) {  
  
    analogWrite(LED2_PIN, pwmValue);  
  
} else {  
  
    // Wenn LED 2 ausgeschaltet ist,  
    // bleibt sie aus  
    analogWrite(LED2_PIN, 0);  
}  
  
// Werte im Seriellen Monitor anzeigen  
Serial.print("POT: ");  
Serial.print(potValue);  
  
Serial.print(" | PWM: ");  
Serial.print(pwmValue);  
  
Serial.print(" | LED1: ");  
Serial.print(led1State);  
  
Serial.print(" | LED2: ");  
Serial.println(led2State);  
  
// Kleine Pause für bessere Lesbarkeit  
delay(200);  
}
```

12. Referenzlösung Challenge

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.4 Challenge: Helligkeit über Seriellen Monitor einstellen

```
// Pinbelegung der ATC-Platine
const int LED1_PIN = 5; // LED 1, PWM-fähig
const int LED2_PIN = 6; // LED 2, PWM-fähig
const int BTN1_PIN = 7; // Taster 1
const int BTN2_PIN = 8; // Taster 2
const int POT_PIN = A0; // Potentiometer, wird hier nicht verwendet
```

```
// Speichert den aktuellen Zustand der LEDs
// false -> LED aus
// true -> LED an
bool led1State = false;
bool led2State = false;
```

```
// Speichert die Helligkeit der LEDs
// Wertebereich: 0 bis 255
int pwmValue = 0;
```

```
// Sperrvariablen für die Taster
// Dadurch wird pro Tastendruck nur einmal geschaltet
bool btn1Gesperrt = false;
bool btn2Gesperrt = false;
```

```
void setup() {

    // LEDs als Ausgänge konfigurieren
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);

    // Taster als Eingänge mit interner Pull-Up-Schaltung
    // nicht gedrückt -> HIGH
    // gedrückt -> LOW
    pinMode(BTN1_PIN, INPUT_PULLUP);
    pinMode(BTN2_PIN, INPUT_PULLUP);

    // Serielle Kommunikation starten
    Serial.begin(9600);

    // LEDs beim Start ausschalten
    analogWrite(LED1_PIN, 0);
    analogWrite(LED2_PIN, 0);

    // Hinweis im Seriellen Monitor ausgeben
    Serial.println("Gib einen Wert zwischen 0 und 255 ein:");
}
```

```
void loop() {  
  
    // Tasterzustände einlesen  
    bool btn1Pressed = digitalRead(BTN1_PIN);  
    bool btn2Pressed = digitalRead(BTN2_PIN);  
  
    // Prüfen, ob im Seriellen Monitor ein Wert eingegeben wurde  
    if (Serial.available() > 0) {  
  
        // Zahl aus dem Seriellen Monitor lesen  
        int neuerWert = Serial.parseInt();  
  
        // Prüfen, ob der eingegebene Wert im erlaubten Bereich liegt  
        if (neuerWert >= 0 && neuerWert <= 255) {  
  
            // Neuen Helligkeitswert speichern  
            pwmValue = neuerWert;  
  
            // Rückmeldung im Seriellen Monitor ausgeben  
            Serial.print("Neue Helligkeit: ");  
            Serial.println(pwmValue);  
  
        } else {  
  
            // Fehlermeldung bei ungültigem Wert  
            Serial.println("Fehler: Wert muss zwischen 0 und 255 liegen.");  
        }  
    }  
  
    // Taster 1 schaltet LED 1 um  
    if (btn1Pressed == LOW && btn1Gesperrt == false) {  
  
        led1State = !led1State;  
        btn1Gesperrt = true;  
    }  
  
    // Sperre wieder freigeben,  
    // sobald Taster 1 losgelassen wurde  
    if (btn1Pressed == HIGH) {  
        btn1Gesperrt = false;  
    }  
  
    // Taster 2 schaltet LED 2 um  
    if (btn2Pressed == LOW && btn2Gesperrt == false) {  
  
        led2State = !led2State;  
        btn2Gesperrt = true;  
    }  
}
```

```
// Sperre wieder freigeben,  
// sobald Taster 2 losgelassen wurde  
if (btn2Pressed == HIGH) {  
    btn2Gesperrt = false;  
}  
  
// LED 1 entsprechend Zustand und Helligkeit schalten  
if (led1State == true) {  
  
    analogWrite(LED1_PIN, pwmValue);  
  
} else {  
  
    analogWrite(LED1_PIN, 0);  
}  
  
// LED 2 entsprechend Zustand und Helligkeit schalten  
if (led2State == true) {  
  
    analogWrite(LED2_PIN, pwmValue);  
  
} else {  
  
    analogWrite(LED2_PIN, 0);  
}  
}
```

13. Abschluss

Diese Programmierübung erweitert dein Verständnis deutlich, da du gelernt hast, Zustände im Seriellen Monitor sichtbar zu machen, analoge Messwerte auszulesen und weiterzuverarbeiten, Wertebereiche mit `map()` umzuwandeln sowie die Helligkeit von LEDs mithilfe von PWM zu steuern.

Zusätzlich konntest du erstmals Eingaben über den Seriellen Monitor verarbeiten und damit wichtige Grundlagen für die Arbeit mit Sensoren, PWM und strukturierter Programmlogik schaffen.

Level 1 ist damit technisch abgeschlossen und bereitet gezielt auf die Inhalte von Level 2 vor.